Document Smart Highlights



1 Version

0.4.0-SNAPSHOT - 6 - 20261020-122319

2 Introduction

Welcome to **DSH - Document Smart Highlights**. Document Smart Highlights aims to provide web services that accept PDF and HTML files and return a list of keywords and the most relevant sentences, extracted with NLP techniques; the sentences through automatic document summarization. More details at the wiki.

What 0.3.0 does today. It accepts a PDF document, stores it in a NoSQL store, puts its id on a queue for indexing, and reports the document's processing status. Nothing consumes that queue yet, so 0.3.0 does not index documents and does not extract keywords or sentences.

This project is distributed as source code; no binary distribution is provided yet. Building it produces the REST API as an executable Spring Boot jar, which runs on its own, with no separate server to install.

The goals for this project are basically two:

1. Implement an NLP based system to extract relevant information from PDF files in a potentially scalable fashion, using a NoSQL store and queues. This infrastructure stores requests and chains a workflow of operations (workers) that extract keywords and relevant sentences, all provided as a REST API.
2. Learn concepts of NLP associated with scaleable cloud based REST API building. The technology used is java based, so as another goal here we can mention the build learning process using Spring Framework in order to build the API and infrastructure.

As part of the goals is learning about technologies, comments and contributions are welcome. However, this is not a final product, application or concept. Just a point for experimentation and proofing.

If anyone out there is interested in contribute or apply this project on a more product-oriented environment, please get in touch through the email: marcelo.riss@gmail.com.

Wiki: https://github.com/MRISS-Projects/dsh/wiki

Project Development Documentation: https://mriss-projects.github.io/dsh-docs/

3 Package/Folders Description

- **dsh-test-dataset**: the PDF files used as fixtures by automated tests.
- **dsh-data**: the data models, **Document**, **Keyword** and **Sentence**, and the workflow of statuses a document moves through while it is processed. Also the document store: an interface, and the in-memory implementation DSH runs with by default.
- **dsh-rest-api**: the application. A Spring Boot jar exposing the REST API: document submission and processing status. Querying the results (keywords and relevant sentences) is not available yet.
- **dsh-solr**: a parent module grouping two Solr plugins, plus a `solrconfig.xml`. Nothing in 0.3.0 runs Solr, and replacing it is proposed in ADR-001.

    - **solr-terms-vector-order**: a search component that returns a document's term vector ordered by term statistics such as term and document frequency.
    - **solr-advanced-numbers-filter**: a token filter for tokens made of digits.
- **dsh-doc-indexer-worker**: a Spring Boot application holding only its main class so far. Its job will be to take a document id off the indexing queue, read the document from the database, send it to the indexer, and split its text into paragraphs, sentences and terms.
- **dsh-doc-analyser**: a parent module grouping the analysis modules below.
    - **dsh-doc-processor-worker**: an empty Spring Boot application for now; the plan is for it to take indexed documents off a queue and run both extractors on them.
    - **dsh-keyword-extractor**: not implemented yet. It will score each term of a document with combinations of TF/IDF and return the best ranked terms as keywords.
    - **dsh-top-sentences-extractor**: not implemented yet; planned to rank sentences with automatic summarization techniques and return the most relevant ones.
- **dsh-coverage-report**: aggregates the test coverage of every module into one report and the coverage badge.

4 Installation

4.1 Pre-requisites

- Java 17 (Temurin)
- Maven 3.9.16
- Docker, to run RabbitMQ, and for the integration build
- RabbitMQ 3, run from the `rabbitmq:3` image

4.2 Installing/Building the Application

4.2.1 Java

Download and Installation

1. Download an Eclipse Temurin JDK 17 from https://adoptium.net/temurin/releases/?version=17.
    1. **IMPORTANT NOTE**: Download and install a **JDK, not a JRE**. On the download page select Version **17 (LTS)**, your operating system and architecture, and the **JDK** package type.
2. Windows
    1. There should be a .msi windows installer. Just follow the instructions.
3. Linux
    1. Download the .tar.gz file. After downloading it, uncompress it at a folder of your preference.
    2. Create a link. Open a command prompt, go to the JDK parent folder (the folder where you extract JDK into), and type:

```
# replace jdk-17.0.20.1+1 with the directory the archive extracted to
ln -s jdk-17.0.20.1+1 java
```

Setting environment variables

Linux

1. Open the file `/home/[YOUR_USER]/.profile`. This file might be hidden. If it does not appear at your home folder, using the file explorer, type `Ctrl+h`. Go to the end of the file and add:

```
export JAVA_HOME=/your/jdk/parent/folder/java
export PATH=$JAVA_HOME/bin:$PATH
```

Windows

1. Open Control Panel go to System, Advanced system settings, `Environment Variables` button.
2. At the System Variables section, click New.
3. Set JAVA_HOME and point to the root of JDK folder.
4. Search for the variable named «Path» in the list, click on it and press Edit.
5. Prepend the value with:

```
%JAVA_HOME%\bin;
```

Verifying the installation

1. Open a command prompt and type:

```
java -version
```

2. The result should be something like:

```
openjdk version "17.0.20.1" 2026-08-18
OpenJDK Runtime Environment Temurin-17.0.20.1+1 (build 17.0.20.1+1)
OpenJDK 64-Bit Server VM Temurin-17.0.20.1+1 (build 17.0.20.1+1, mixed mode, sha
```

4.2.2 Maven

1. Download maven **3.9.16** from https://archive.apache.org/dist/maven/maven-3/3.9.16/binaries/apache-maven-3.9.16-bin.zip
2. Unzip it on a folder of your preference
3. Set environment variables.
    1. Linux
        1. Put it at your `$HOME/.profile` file

```
export M2_HOME=/path/to/where/you/extracted/maven/apache-maven-3.9.16
export PATH=$M2_HOME/bin:$PATH
export MAVEN_OPTS='-Xmx1024m'
```

        2. If you already have java set up, your `.profile`, it should look like this:

```
export JAVA_HOME=/your/jdk/parent/folder/java
export M2_HOME=/path/to/where/you/extracted/maven/apache-maven-3.9.16
export PATH=$JAVA_HOME/bin:$M2_HOME/bin:$PATH
export MAVEN_OPTS='-Xmx1024m'
```

3. Logout and login again.
4. Test by opening a terminal and typing:

```
mvn -version
```

5. The result should be similar to:

```
Apache Maven 3.9.16 (2bdd9fddda4b155ebf8000e807eb73fd829a51d5)
Maven home: /home/[YOUR_USER]/apps/apache-maven-3.9.16
Java version: 17.0.20.1, vendor: Eclipse Adoptium, runtime: /home/[YOUR_
Default locale: en_US, platform encoding: UTF-8
OS name: "linux", version: "6.8.0-45-generic", arch: "amd64", family: "u
```

2. Windows

1. Open Control Panel go to System, Advanced system settings, **Environment Variables** button.
2. At the System Variables section, click New.
3. Set `M2_HOME` and point to the root of maven folder.
4. Search for the variable `Path` in the list, click on it and press Edit.
5. Put the maven bin folder right after java home:

```
%JAVA_HOME%\bin;%M2_HOME%\bin;
```

6. Set the variable `MAVEN_OPTS`.

```
MAVEN_OPTS=-Xmx1024m
```

7. The result should be similar to:

```
Apache Maven 3.9.16 (2bdd9fddda4b155ebf8000e807eb73fd829a51d5)
Maven home: C:\data\apache-maven-3.9.16
Java version: 17.0.20.1, vendor: Eclipse Adoptium, runtime: C:\Program F
Default locale: en_US, platform encoding: Cp1252
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"
```

4.2.3 RabbitMQ

Run it from the image the integration build is verified against:

```
docker run -d --name dsh-rabbitmq -p 127.0.0.1:5672:5672 rabbitmq:3
```

RabbitMQ needs no set-up: the application connects as its default `guest` user. Stop it with `docker stop dsh-rabbitmq`, and start it again with `docker start`.

4.2.4 Building From Sources

1. Configure Maven's user settings as described in Maven Settings for GitHub Packages below, which is what lets Maven resolve the parent POM `com.mriss.mriss-parent:products`.
2. At the root dsh folder type:

```
mvn clean install
```

This runs the unit tests only, and needs no RabbitMQ running. The integration tests run with `mvn clean install -DintegrationTests`, which starts its own RabbitMQ container, so it needs a running Docker daemon.

5 Configuration

5.1 Maven Settings for GitHub Packages

The parent POM `com.mriss.mriss-parent:products` is published to GitHub Packages, so no build of this repository resolves until your `~/.m2/settings.xml` authenticates against that registry. Three server ids and one property are needed — the same four things that `.github/workflows/ci.yml` writes into its own `settings.xml`:

Entry	Used for
server `MRISS-Projects-maven-repo`	resolving the parent POM and other MRISS artifacts
server `MRISS-Projects-maven-repo-plugins`	resolving MRISS Maven plugins
server `github.com`	`maven-scm-plugin`, when a `-Ddeployment` build commits the regenerated `README.md`
property `github.personal.token`	`maven-changes-plugin`, which reads the closed milestone issues that go into the generated `README.md`. It is a Maven property, not a server credential — the `readme-generation` profile in `parent-poms` wires it into the plugin's `personalToken` parameter — so **omitting it fails silently**: `failOnError` is `false`, the issue list is never produced, and the generated `README.md` comes out with an empty Release Notes section

The credential is one and the same classic personal access token, carrying `read:packages` and nothing else, issued from an account with read access to the `MRISS-Projects` organisation. This repository is public, so no issue-reading scope is required on top. Paste it into all four places in your

own `~/.m2/settings.xml`; it is never committed to this repository. The CI side of the same token is described under Secrets in `docs/devops/README.md`.

```
<settings>
  <servers>
    <server>
      <id>MRISS-Projects-maven-repo</id>
      <username>YOUR-GITHUB-USERNAME</username>
      <password>YOUR-PACKAGES-READ-TOKEN</password>
    </server>
    <server>
      <id>MRISS-Projects-maven-repo-plugins</id>
      <username>YOUR-GITHUB-USERNAME</username>
      <password>YOUR-PACKAGES-READ-TOKEN</password>
    </server>
    <server>
      <id>github.com</id>
      <username>YOUR-GITHUB-USERNAME</username>
      <password>YOUR-PACKAGES-READ-TOKEN</password>
    </server>
  </servers>
  <profiles>
    <profile>
      <id>github-packages</id>
      <properties>
        <github.personal.token>YOUR-PACKAGES-READ-TOKEN</github.personal.token>
      </properties>
      <repositories>
        <repository>
          <id>MRISS-Projects-maven-repo</id>
          <url>https://maven.pkg.github.com/MRISS-Projects/maven-repo</url>
          <releases><enabled>true</enabled></releases>
          <snapshots><enabled>true</enabled></snapshots>
        </repository>
      </repositories>
      <pluginRepositories>
        <pluginRepository>
          <id>MRISS-Projects-maven-repo-plugins</id>
          <url>https://maven.pkg.github.com/MRISS-Projects/maven-repo</url>
          <releases><enabled>true</enabled></releases>
          <snapshots><enabled>true</enabled></snapshots>
        </pluginRepository>
      </pluginRepositories>
    </profile>
  </profiles>
  <activeProfiles>
    <activeProfile>github-packages</activeProfile>
  </activeProfiles>
</settings>
```

6 Usage

With RabbitMQ running and the build done, start the application in either of two ways.

6.1 Running the Jar

```
java -jar dsh-rest-api/target/dsh-rest-api-<version>.jar
```

The RabbitMQ address can be overridden when starting it:

```
java -jar dsh-rest-api/target/dsh-rest-api-<version>.jar \
  --spring.rabbitmq.host=localhost --spring.rabbitmq.port=5672
```

6.2 Running with the Spring Boot Maven Plugin

From the `dsh-rest-api` folder:

```
mvn spring-boot:run
```

Either way, the application is up when the log shows a line like:

```
c.m.dsh.restapi.DshRestApplication - Started DshRestApplication in 9.385 seconds
```

6.3 Where Documents Are Kept

DSH keeps documents in memory. Nothing has to be installed for it, and every document is lost when the application stops: a run with no profile is for trying DSH out and for tests, not a deployment. The store is selected by `dsh.persistence`. Its default, and its only value until the Firestore store arrives, is `memory`. Any other value stops the startup.

6.4 Calling the API

The REST API has the two operations of the document resource:

- **submit**, `POST /v1/dsh/document/submit`: uploads a PDF file (`contents`) with a `title`, and returns a token. If the submission fails, the token is `ERROR` and the message says why.
- **status**, `GET /v1/dsh/document/status/{token}`: returns the processing status of the document the token was issued for. Right after a submit it can show `QUEUED_FOR_INDEXING` for a moment; in 0.3.0 a document then ends at `QUEUED_FOR_INDEXING_SUCCESS`: stored, and its id queued for indexing. It shows `QUEUED_FOR_INDEXING_ERROR` if the id could not be queued, and an unknown token returns `TOKEN_NOT_FOUND`. No later status is reached until the indexing worker exists.

Submit a PDF file, then ask for its status with the token the submit returned:

```
curl -F title="My document" -F contents=@/path/to/file.pdf \
  http://localhost:8080/v1/dsh/document/submit
curl http://localhost:8080/v1/dsh/document/status/<token>
```

The API description, in Swagger 2 format, is served at `http://localhost:8080/v2/api-docs?group=dsh-app`. The Swagger UI is not served in 0.3.0; see #144.

7 Release Notes

7.1 Version 0.4.0-SNAPSHOT

#	Type	Summary	Assignee	Reporter	Updated
173	task	Record the reference papers Wave 3 is built from, and keep their full text out of git unless a licence allows it	null	mriss	10/16/26
162	task	Make the OpenAPI document and the feature specs describe the API that exists	null	mriss	10/16/26
171	bug	NOT_INDEXED_E answers DEQUEUED_FOR description and message	null	mriss	10/16/26
160	task	[STORY] ADR-003: decide the document pipeline, and revise ADR-001 to match	null	mriss	10/13/26
48	task	Investigate how to use profiles (dev, stagging, production) with spring and maven	null	mriss	10/10/26

128	task	Point the wiki's release link at releases/products/dsh once 0.3.0 is released	null	mriss	10/6/26
152	task	[STORY] Generate the document-status diagram from DocumentStatus and fail the build when it drifts	null	mriss	10/6/26
150	task	Re-pin the parent to the released parent-poms 3.10.0, and retire what the SNAPSHOT pin needed	null	mriss	10/3/26
127	enhancement	Deploy the snapshot site from DEVELOP, as parent-poms' deploy.yml does	null	mriss	10/2/26

7.2 Version 0.3.2

#	Type	Summary	Assignee	Reporter	Updated
146	bug	[STORY] Release DSH 0.3.1 with a release site that carries its test reports and coverage badge	null	mriss	10/3/26

7.3 Version 0.3.1

#	Type	Summary	Assignee	Reporter	Updated
-	-	No issues	-	-	-

7.4 Version 0.3.0

#	Type	Summary	Assignee	Reporter	Updated

143	task	Make the README and the site index describe what DSH does today	null	mriss	9/29/26
139	bug	dsh-data connects to MongoDB unauthenticated, with connection settings fixed at build time	null	mriss	9/29/26
46	task	Implement integration tests using embedded tomcat server.	null	mriss	9/29/26
113	task	Remove the dead ‘main’ branch trigger from api-testing.yml and documentation-sync.yml	null	mriss	9/28/26
115	bug	[STORY] version.properties ships an unresolved ${jenkins.build.nur in two modules	null	mriss	9/28/26
114	task	Release and hotfix wrappers do not supply the build properties DSH's reactor needs	null	mriss	9/28/26
122	task	Close the file streams that test fixtures leave open	null	mriss	9/28/26
124	task	[STORY] Keep dsh-test-dataset fixtures and test classes out of production artifacts	null	mriss	9/28/26
43	task	configure surefire, jacoco and other usefull reports for the maven generated docs	null	mriss	9/28/26

104	task	Regenerate the coverage badge, or stop publishing a stale one	null	mriss	9/27/26
70	bug	Project link not working at maven generated site.	mriss	mriss	9/27/26
90	bug	index.html missing from published site on gh-pages (root + all submodules)	null	mriss	9/27/26
87	task	Update Maven pinned version from 3.9.9 to 3.9.16 in documentation and GitHub Actions	null	mriss	9/26/26
123	task	[STORY] Stop passing the Mongo setup inputs to project-staging.yml	null	mriss	9/26/26
112	task	Reclassify the Spring-context tests as integration tests and pay the unit-coverage bill	null	mriss	9/26/26
117	task	Pass development_brar to the release and hotfix wrappers	null	mriss	9/25/26
111	task	[STORY] Let release.yml and hotfix.yml dispatch a release rehearsal	null	mriss	9/23/26
94	task	Make check-spec-references enforcing, or remove it	null	mriss	9/19/26
85	task	Update documentation: replace Maven 3.3.9 with 3.9.9 and standardise Java version to 17	null	mriss	9/19/26

86	task	Pin Maven 3.9.9 in all GitHub Actions workflows that invoke Maven	null	mriss	9/18/26
97	task	[STORY] Resolve the tooling orphaned by the Travis estate removal	null	mriss	9/18/26
101	task	[STORY] Fail CI when the package token cannot authenticate, not just when it is absent	null	mriss	9/18/26
103	task	Make PR review rounds repo-aware and authoritative	null	mriss	9/17/26
93	task	Remove the redundant coverage ratchet - jacoco:check at 95% is already inherited	null	mriss	9/17/26
99	task	[STORY] Standardise Maven builds on -U while the parent is a SNAPSHOT	null	mriss	9/17/26
95	bug	Use a read-only token for CI package authentication	null	mriss	9/17/26
92	task	Remove the dead Travis build estate	null	mriss	9/17/26
84	task	[FEATURE] Unify/reorg of deploy/release profiles	mriss	mriss	5/22/26
72	task	Refactor all artifactId names to be lower case to be in maven naming standards	mriss	mriss	5/22/26

83	enhancement	[FEATURE] Adapt DSH to the new parent poms version 3.8.0	mriss	mriss	4/23/26
68	wontfix	Update readme file with instructions on how to install SOLR.	mriss	mriss	4/23/26
67	wontfix	Implement configuration class and setup as a daemon.	mriss	mriss	4/23/26
66	wontfix	Configure and test OpenNLP POS filter.	mriss	mriss	4/23/26
13	wontfix	Create and test SOLR DAO	mriss	mriss	4/23/26
71	task	Adapt pom structure to new parent poms.	mriss	mriss	4/13/26
14	task	Install and get trained on SOLR tutorial	mriss	mriss	2/22/20

7.5 Version 0.2.4

#	Type	Summary	Assignee	Reporter	Updated
64	bug	Test code report is being generated with 0 tests.	null	mriss	4/26/19

7.6 Version 0.2.3

#	Type	Summary	Assignee	Reporter	Updated
63	bug	Attach jacoco badge generation at verify phase is generating badge with 0%	mriss	mriss	4/26/19

7.7 Version 0.2.2

#	Type	Summary	Assignee	Reporter	Updated
61	enhancement	Add jacoco badge	mriss	mriss	4/14/19
60	enhancement	Add travis badge.	mriss	mriss	4/14/19
59	enhancement	Add jacoco coverage plugin and report.	mriss	mriss	4/12/19

7.8 Version 0.2.1

#	Type	Summary	Assignee	Reporter	Updated
58	bug	stage is being executed at master during release process	mriss	mriss	4/11/19

7.9 Version 0.2.0

#	Type	Summary	Assignee	Reporter	Updated
40	task	Add DSH to travis CI following settings from changes plugin.	mriss	mriss	4/11/19
39	task	Publish dsh site on gh-pages branch instead of another repo.	null	mriss	3/18/19
41	task	Move project from organization to the git project dsh. Change next milestone to 0.2.0	mriss	mriss	3/18/19

7.10 Version 0.0.1

#	Type	Summary	Assignee	Reporter	Updated

32	task	Configure DSH to use git as scm tool and proceed to release.	mriss	mriss	3/15/19
38	task	Configure distribution management to local nexus and test snapshot deploy with deployment profile.	mriss	mriss	3/1/19
37	task	Replace release notes and release history properties using deployment profile.	mriss	mriss	10/14/18
36	task	Replace version property ad readme and commit using deployment profile	mriss	mriss	2/19/18
35	task	Test maven site publication using github using deployment profile.	mriss	mriss	2/15/18
34	task	Configure changes plugin and changes report to use github issues and test maven site generation.	mriss	mriss	2/12/18
33	task	Configure maven scm to use git	mriss	mriss	2/12/18
10	task	Test rest API module inside tomcat server inside eclipse as a war distribution.	mriss	mriss	2/11/18
30	task	Configure Swagger	mriss	mriss	2/4/18
11	task	Create and test rest service layer:	mriss	mriss	1/16/18

23	task	Add message and error handling for RabbitMQ queue submission.	mriss	mriss	1/9/18
22	task	Create document submission workflow	mriss	mriss	1/6/18
28	task	Create document status enumeration and define workflow transition and validation class.	mriss	mriss	1/6/18
27	task	Add extra columns at the Document model class for status description and status message.	mriss	mriss	1/6/18
25	task	Test message sending exception.	mriss	mriss	1/4/18
17	task	Create web service logic to generate token and return it while starting the document storage at mongo asynchronously.	mriss	mriss	1/3/18
9	task	Create services:	mriss	mriss	12/19/17
21	task	Create mongodb storage service.	mriss	mriss	12/19/17
20	task	Update documentation with RabbitMQ installation.	mriss	mriss	12/19/17
18	task	Create logic to enqueue the mongo document id to RabbitMQ using Spring integration example app.	mriss	mriss	12/18/17
19	task	Feature/mongo dao	mriss	mriss	12/10/17

15	task	Create and test MongoDAO	null	mriss	12/10/17
16	task	Test models	mriss	mriss	12/9/17
12	task	Create dsh-test-dataset module having all PDF and HTML files used for testing.	mriss	mriss	12/8/17
8	task	* Create model for the documents with following columns:	mriss	mriss	12/7/17
7	task	* Create package structure	null	mriss	12/7/17
3	task	Create parent pom	mriss	mriss	12/7/17
5	task	Create a model module to have the model classes of keywords, sentences and documents.	mriss	mriss	12/7/17
6	task	Organize dependency management among modules.	mriss	mriss	12/7/17
2	task	Create project structure using spring boot	mriss	mriss	12/7/17
4	task	Complete project structure	mriss	mriss	12/7/17
1	task	Install RabbitMQ	mriss	mriss	11/29/17